2016 EXAMINATIONS

Lancaster University

Part II

COMPUTING AND COMMUNICATIONS [2 Hours]

SCC.211 Operating Systems

Candidates are asked to answer THREE questions from FOUR; each question is worth a total of 20 marks.

Use a separate answer book for each question.

Question 1

On the following page you will find C-based pseudocode of a non-preemptive, user-level thread library. This offers four API calls: *thread_init()* to initialise the library; *thread_create()* to create a new thread; *thread_yield()* to allow the calling thread to indicate that it is willing for other threads to take their turn to run at this point; and *thread_exit()* to enable the calling thread to destroy itself.

1.a Explain why it is necessary for a newly-created thread to have a new stack (see 3rd line in *thread_create()*).

[**2 marks**]

1.b From the pseudocode you can see that, a thread's *context* comprises storage for the stack pointer and the program counter. Explain the notion of context in a more general sense, and explain why we need to deal with context in a thread library such as this.

[**2 marks**]

1.c Describe the behaviour of the thread library when the *context_save()* call in *thread_init()* is reached (hint: there are two cases to consider).

[**4 marks**]

1.d Explain why execution never reaches the point labelled /* NOTREACHED */ in *thread_yield()*. Then explain the significance of this for the functioning of the thread library.

[**4 marks**]

1.e Provide pseudocode for the missing *thread_exit()* function (you will notice that this is called from *thread_init()*). Be careful to write your pseudocode in the same style as the rest of the library, and make sure you make proper reference to the relevant data structures.

[**8 marks**]

C-based pseudocode for question 1 appears on next page

Total 20 marks

C-based pseudocode for question 1

```
/****** FILE: thread.h for question 1 ******/

typedef struct queue {/* detail suppressed */} Queue;
typedef struct context {
  void *stack_pointer;
  void *program_counter;
} Context;
typedef struct thread {
  Context *c;
  void (*func)(int a);
  int  arg;
} Thread;

/* context_restore(): restore a previously-saved context, *c */
context_restore(Context *c);

/* context_save(): save current context to *c;
 * return 0 on initial return;
 * return 1 on subsequent returns caused by calls of context_restore(c);
 */
context_save(Context *c);
```

```
/****** FILE: thread.c for question 1 ******/

Thread *current_thread; /* pointer to the currently running thread */
Queue   run_queue, recycle_queue;
Context start_context;

void thread_init()
{
  if (context_save(&start_context) == 1) {
    (*(current_thread->func))(current_thread->arg);
    thread_exit();
    /* NOTREACHED */
  }
}
void thread_create(void (*func)(int a), int arg)
{
  Thread *t = allocate a new Thread struct;
  t->func = func; t.arg = arg;
  t->c.stack_pointer = malloc a new stack;
  t->c.program_counter = start_context.program_counter;
  put t on run_queue;
  thread_yield();
}
void thread_yield()
{
  if (context_save(current_thread->c) == 0) {
    put *current_thread on run_queue;
    current_thread = call scheduler to get a new Thread to run;
    context_restore(current_thread->c);
    /* NOTREACHED */
  }
  take Thread structs off recycle_queue and free them and their stacks;
}
void thread_exit() { /* To be completed by you. */ }
```

Question 2

2.a If you were asked to write a new implementation of *spin locks*, would it make sense to use an existing *semaphore* implementation as a basis to help you do this? Please answer yes or no.

If you answered *yes*, explain why and how semaphores can usefully be employed in the implementation of spin locks.

If you answered *no*, explain why you think it would make no sense to implement spin locks in terms of semaphores.

[**3 marks**]

2.b If you were asked to write a new implementation of *semaphores*, would it make sense to use an existing *spin lock* implementation as a basis to help you do this? Please answer yes or no.

If you answered *yes*, explain why and how spin locks can usefully be employed in the implementation of semaphores.

If you answered *no*, explain why you think it would make no sense to implement semaphores in terms of spin locks

[**3 marks**]

2.c Define *mutual exclusion* and *condition synchronisation*, making sure that your definitions clearly distinguish the two concepts. Give a pseudocode-based example of each

[**4 marks**]

Question 2 continues on next page…

Question 2 continued...

2.d The following is a Java class that implements a concurrently-accessible *bounded counter* that operates over the range 0..100. The *increment()* operation increments the counter, unless it is already at the 'bounding condition' of 100, in which case *increment()* blocks until the counter has become less than 100, and then increments it. The *decrement()* operation is intended to work analogously, with its corresponding 'bounding condition' occurring when the counter is at 0.

```
public class BoundedCounter {
  long count = 0;

  public synchronized long value() {return count;}

  public synchronized void increment()
  {
    if (count == 100) wait();
    count = count + 1;
    if (count == 1) notify();
  }

  public synchronized void decrement()
  {
    /* To be completed by you. */
  }
}
```

i. Identify which of the Java features present in this code relate to mutual exclusion, and which relate to condition synchronisation.

[2 marks]

ii. Complete the *decrement()* operation.

[2 marks]

iii. The *notify()* method used in our BoundedCounter unblocks a thread that had previously been blocked on a corresponding call of *wait()*. Java supports a related method – *notifyAll()* – that unblocks all threads that had previously blocked on a corresponding call of *wait()*. Rewrite the BoundedCounter class to use *notifyAll()* and *wait()* instead of *notify ()* and *wait()*.

[4 marks]

iv. Discuss in general terms the pros and cons of using *notifyAll()* and *wait()* instead of *notify()* and *wait()*.

[2 marks]

Total 20 marks

Question 3

3.a Why are the *Hierarchical Paging* systems that have been widely used in computer systems becoming a problem with increasing memory sizes?

[**2 marks**]

3.b Identify the role of a *Translation Lookaside Buffer (TLB)*, produce a diagram that clearly shows how a TLB would be used, and explain why and how a TLB is managed at the process level. Your answer must identify any error states and what should be done if these occur.

[**6 marks**]

3.c An important consideration when managing processes is the *working set*

i. Explain what we mean by the *working set* of a process, outline how page faults could be used to tune the working set of a process, and how getting this wrong can lead to *thrashing*.

ii. What is thrashing, and what would you expect to see when it is occurring?

[**5 marks**]

3.d The following diagram represents the state of an 8KiB (8 x 1024 bytes) memory area managed by a *buddy allocator* following requests for four 1KiB memory regions.

8KiB

A 1KiB	Free 1KiB	B 1KiB	C 1KiB	Free 4KiB

Produce a sequence of diagrams that show the free and allocated regions in this 8KiB memory region and their size, in the format shown above, after the following sequence of allocate and free requests.

Free (B); Free (C); B = allocate (5KiB); Free (A); A = allocate (3KiB); B = allocate (1KiB)

Your solution **must** always find and allocate the leftmost free memory area, subject to the normal rules of buddy allocation.

[**7 marks**]

Total 20 marks

Question 4

4.a Operating systems provide abstractions to applications and themselves make use of hardware abstractions through a *Hardware Abstraction Layer*. Giving an example of each type of abstraction, briefly outline the benefits such abstractions offer both operating systems and the applications and services these operating systems support.

[**4 marks**]

4.b A number of the Intel x86 processors include support for four ring levels. What are these and why do many operating systems make use of only two of these rings?

[**2 marks**]

4.c Filesystems such as those commonly used in Unix systems support the creation of links to existing files.

- Compare and contrast hard links and symbolic links being careful to identify the different ways in which each can be used.
- Outline any restrictions on the use of these mechanisms along with the reasons for these restrictions.
- Identify any information that must be stored, along with its location, for the correct operation of each link mechanism.

[**6 marks**]

4.d In common Unix filesystems such as ext2 the storage of files is subject to internal fragmentation but the directory system suffers both *internal fragmentation* and *external fragmentation*.

- Outline both these different forms of fragmentation being careful to highlight the differences between them, the type of allocation scheme that leads to each form of fragmentation, and how easy it might be to reclaim fragmented space.
- Then, clearly identifying the data structures involved, explain where fragmentation in directories and file storage is seen and the reasons such fragmentation occurs.

[**8 marks**]

Total 20 marks

--- END OF PAPER ---